

MORIA: Mining the Cache for Miss Parallelism

Marina Vemou*

Alexandros Daglis^{†*}

^{*}*School of Computer Science, Georgia Institute of Technology, Atlanta, GA, USA*

[†]*School of Informatics, University of Edinburgh, Edinburgh, Scotland, UK*

Abstract—Memory-level parallelism is a primary determinant of single-thread CPU performance, yet it is limited by the number of Miss Status Handling Registers (MSHRs) in L1 caches. While the bandwidth-delay product of increasingly deep and complex memory systems continues to grow, L1 MSHR provisioning remains modest, causing frequent MSHR exhaustion and stalling the CPU even in the presence of available memory bandwidth. We find that modern workloads routinely exceed available MSHR capacity, exposing MSHRs as a critical and growing bottleneck.

We propose MORIA, a technique that overcomes this limitation by virtualizing MSHRs within a minimally augmented L1 cache. MORIA dynamically repurposes cache lines to hold miss-tracking state when dedicated MSHRs are depleted, leveraging the high associativity and underutilized capacity of modern caches to provide scalable, on-demand miss handling. We present and evaluate two MORIA variants: a *hybrid*, which augments conventional MSHRs with an in-cache fallback, and a *full in-cache*, which eliminates dedicated MSHRs. Across 86 workloads, MORIA improves performance by 1.14 $\times$, on average, by eliminating MSHR-induced stalls, with modest area and energy overheads.

Index Terms—Miss parallelism, in-cache MSHR

I. INTRODUCTION

Growing computational demands have driven the design of increasingly large manycore CPUs. As a result, the memory subsystem scales in both bandwidth and capacity to keep pace. The combined effect of these trends is growing effective memory access latency, often accompanied by higher variability due to deep and complex memory hierarchies. To illustrate, unloaded memory access latency has gradually almost doubled to ~ 120 ns, while multi-socket and tiered memory systems contribute the additional challenge of 1.5–5 $\times$ latency variability.

Miss Status Handling Registers (MSHRs) are a crucial microarchitectural component that enables high-performance non-blocking caches by allowing multiple outstanding memory accesses. To sustain high memory-level parallelism (MLP), the number of MSHRs must scale with the bandwidth-delay product of the memory system. As this product increases in modern systems, so does the demand for MSHR capacity. However, despite this trend, MSHR provisioning at the L1 cache level remains modest in modern CPUs.

This mismatch between MSHR demand and provisioning is increasingly problematic. We provide empirical evidence that the number of concurrent primary and secondary misses often exceeds the available MSHR capacity, particularly in memory-intensive workloads, to the detriment of single-thread performance. As a result, CPU cores frequently stall due to MSHR exhaustion, limiting achievable MLP and degrading

overall performance. While modern CPUs typically supply 16–24 L1 MSHRs with 8 subentries each, we find that many workloads exhibit significantly higher demand of up to 111 and 210, respectively.

The seemingly simple solution of provisioning more MSHRs is neither cheap nor straightforward. MSHRs are highly associative structures, and aggressively over-provisioning them to account for worst-case bandwidth-delay occurrences incurs significant area and energy overheads for limited average-case benefit. Moreover, the design of CPU microarchitecture is often loosely coupled to the memory system beyond the cache hierarchy, making precise MSHR provisioning at CPU design time for all possible operating conditions impractical.

In this work, we propose MORIA (MSHR On-demand Reservation with In-cache Allocation), a hardware mechanism that virtualizes MSHR functionality within the L1 cache itself, enabling additional miss-handling capacity with minimal hardware modifications. The key insight behind MORIA is that modern L1 caches are highly associative structures with underutilized capacity in many execution phases, which can be opportunistically repurposed to hold additional MSHR state when dedicated MSHRs are exhausted. Furthermore, the large size of the cache’s data array entries (typically 64B) can naturally accommodate many subentries per MSHR, which is highly beneficial for certain workloads, like graph processing.

Our work revisits earlier proposals for in-cache MSHRs explored in the 90s [19], which were ultimately set aside in favor of dedicated MSHR structures. However, modern cache designs differ substantially from those earlier systems: contemporary L1 caches are predominantly 8–12-way associative and feature wider datapaths and more flexible control logic. These advances fundamentally change the design tradeoffs, motivating a re-examination of in-cache MSHR techniques under modern constraints.

We present two embodiments of MORIA and analyze their tradeoffs: a *hybrid* design (MORIA-h) that uses in-cache MSHRs as a fallback when dedicated MSHRs are exhausted, and a *full* design (MORIA-f) that reveals the potential of completely eliminating dedicated MSHRs. We evaluate both variants and show that MORIA improves performance by alleviating MSHR-induced stalls, while incurring negligible area and modest energy overheads.

In summary, we make the following contributions:

- We revisit the potential of in-cache MSHRs under modern technological realities, and provide extensive evidence that they represent a flexible mechanism that matches the

growing and variable MSHR needs of current systems and workloads.

- We devise MORIA, a mechanism that augments a standard cache with the ability to repurpose its data entries dynamically to hold MSHR information, while only requiring minimal hardware modifications.
- We design, evaluate, and analyze the tradeoffs of two MORIA embodiments. MORIA-h *augments* and MORIA-f *replaces* regular MSHRs with in-cache MSHRs, respectively.
- Our evaluation using 86 workloads across six benchmark suites demonstrates average performance gains of $1.14\times$ and up to $2.37\times$.

II. BACKGROUND AND MOTIVATION

A. Miss Handling Architectures

Modern CPUs mitigate the memory wall by exploiting Memory-Level Parallelism (MLP) to handle multiple memory requests concurrently. Supporting this concurrency requires extensive resources in both the execution pipeline, such as deep instruction windows and Load-Store Queues (LSQs), and the memory hierarchy, particularly Miss Status Handling Registers (MSHRs).

MSHRs were first introduced in the 80s [30], [47], as a mechanism to enable non-blocking caches—i.e., prevent a cache miss from blocking subsequent accesses and allow multiple concurrent misses; a crucial requirement for high-performance memory systems. An MSHR keeps track of a pending cache miss that has not yet been serviced by the lower levels of the memory hierarchy.

Fig. 1 presents an example MSHR implementation. MSHRs are typically organized as a fully associative structure. Each MSHR stores the line address of the request, as well as information about the request’s origin (equivalently, the response’s destination). A single MSHR typically features several *subentries* to track multiple requests for the same line. Each MSHR subentry must track the word offset, information about the request’s origin (or requester’s ID), such as the LSQ entry that triggered the miss, as well as other information about the request, such as its type (read or write).

The MSHR array is accessed on cache miss and on cache fill. On a miss, the array is searched associatively to determine whether there is a prior pending miss to the same cache line; if not, a new MSHR entry is allocated. Misses that allocate a new MSHR entry in the array are *primary* misses. A hit in the MSHR array implies an already pending miss to the same cache line. Misses that hit in the MSHR array are *secondary* misses and only allocate a new subentry in the MSHR that is already allocated for the same cache line. When data returns from lower levels of the memory hierarchy, the information stored in the MSHR subentries is used to forward the data to all waiting requesters in the CPU’s LSQ.

When MSHR or subentry capacity is depleted, further requests to the cache are blocked until the required miss-handling resources become available. Therefore, MSHR provisioning has significant performance implications. Fundamen-

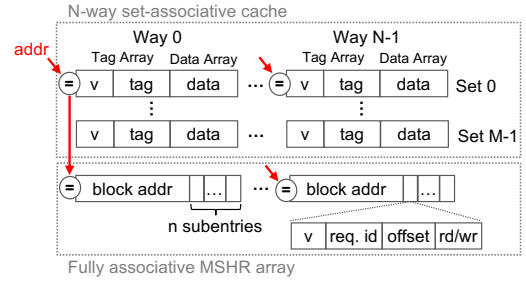


Fig. 1: Baseline cache and MSHR architectures.

tally, such provisioning can be guided by queuing theory: as per Little’s Law, to fully utilize available bandwidth, a CPU must support enough in-flight misses to cover the memory access round-trip latency. Assuming the existence of sufficient MLP in an executing program, fully utilizing the memory system’s capabilities requires provisioning MSHRs proportional to the product of the memory bandwidth and access latency. An undersized MSHR array caps a CPU’s ability to expose sufficient MLP, leading to underutilized bandwidth, pipeline stalls, and performance loss.

However, scaling the MSHR array proportionately with the growing parallelism capabilities of aggressive manycore CPUs and the growing bandwidth-delay product of memory systems is challenging. MSHR implementations are typically based on fully associative structures, searched on every cache miss and fill. As their power consumption scales linearly with their entries, simply scaling them to accommodate worst-case latencies significantly increases overall power consumption and silicon real estate demand.

To improve MSHR scalability, Tuck et al. [50] propose a hierarchical organization that sustains high MLP without the prohibitive power and timing penalties of a single, large fully associative array. Nevertheless, the hardware overhead and complexity of maintaining large-scale associative lookups remain limiting factors in practice. Consequently, most commercial CPUs continue to offer limited MSHRs at the L1 cache. To illustrate, each L1 cache in Intel Sapphire Rapids and AMD Zen 4 can track up to 16 and 24 outstanding misses, respectively [5], [6], [7]. While such numbers suffice on average, they are well below the peak demand of several bandwidth-intensive workloads, as we quantify in §II-C. As a result, MSHR limitations cap the concurrency of execution phases with high inherent MLP availability, hurting single-thread performance.

B. Technology Trends Pushing MSHR Demand

1) *Larger System Scale Compounds Memory Access Latency*: The memory access latency experienced by a core is increasingly less dictated by the memory controller and device, and more by the cache hierarchy and interconnect fabric traversal. Fig. 2 shows how the industry-wide transition toward massive core counts has led to a gradual increase in memory access latency. Earlier monolithic designs such as Intel’s Westmere featured up to 6 cores with a 67ns load-to-use (unloaded) latency [2], which grows to 112–124ns

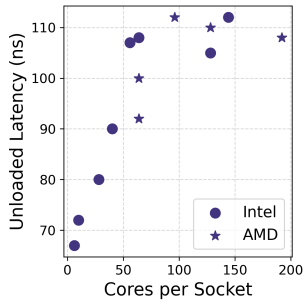


Fig. 2: Average unloaded memory access latency of various modern AMD EPYC and Intel Xeon CPUs.

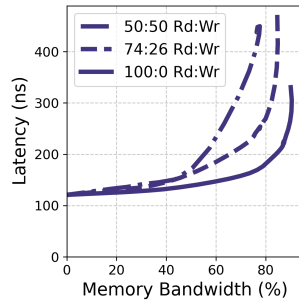


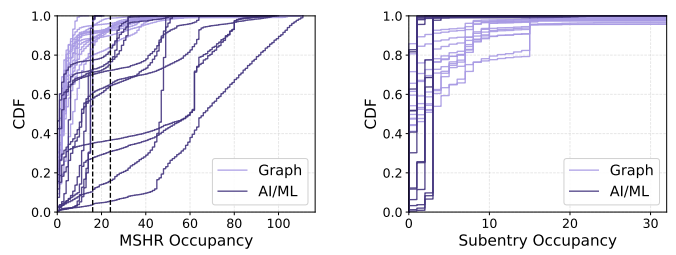
Fig. 3: Memory access latency for a DDR-based system at different utilization points and Rd:Wr ratios [17].

in modern multi-chiplet CPUs like Sierra Forest [3]. Such latency increase is the price of scalability, a tradeoff justified by major gains in throughput, compute density, and resource consolidation.

2) *Complex Memory Hierarchies Amplify Memory Access Latency Variability*: As data-intensive applications (e.g., large-scale graph processing, real-time analytics) outpace the capacity of local DRAM, memory expansion techniques have been in high demand. Distributed shared memory machines have been in use for decades for this purpose. In such multi-socket systems, each CPU has its own local memory, but can also directly access other CPUs’ memory over a coherent interconnect, such as Intel’s Ultra Path Interconnect or AMD’s Infinity Fabric. While enabling a multiplicative increase in total addressable memory, such systems also introduce Non-Uniform Memory Access (NUMA) latency, as accesses to a remote socket’s memory must traverse the coherent interconnect. Such NUMA effects result in remote memory access latencies of $1.5\text{--}2\times$ [31]—or even up to $4\text{--}5\times$ for “big iron” machines with 16+ sockets [11], [15], [24]—higher than local.

More recently, the Compute Express Link (CXL) technology has attracted substantial interest as a way to populate additional memory in a disaggregated fashion, expanding both the memory capacity and bandwidth a CPU can tap into. By utilizing the CXL.mem protocol, a CPU can directly access a CXL-attached memory device as part of the system’s physical address space. The additional capacity and bandwidth afforded by CXL comes at the cost of considerable latency penalty, which can drastically vary by device and configuration, ranging from $2\times$ to $5\times$ of local DRAM access [49]. Consequently, whether CXL-based memory is used as a replacement [16] or an augmentation [27], [35] of a typical DDR-based memory system, the memory system’s bandwidth-delay characteristics, and hence its attached CPU’s optimal MSHR provisioning, is significantly affected.

While the same CPU product lines may be deployed in a variety of large-scale multi-socket or CXL-enhanced memory systems, CPU microarchitecture designers provision hardware resources (including MSHRs) for a typical deployment scenario. Without tailored MSHR sizing, deployments with



(a) MSHR demand. Vertical lines indicate MSHR capacity for Intel Sapphire Rapids (left) and AMD Zen 4 (right). (b) Subentries per MSHR demand. Modern CPUs commonly feature 4–8 subentries per MSHR.

Fig. 4: Demand of Graph and AI/ML workloads on MSHRs and their subentries, recorded on a simulated system with infinite MSHRs and unbounded subentries.

higher-latency memory systems are more likely to encounter MSHR pressure and performance loss due to hardware-induced MLP restrictions.

3) *System Load Dictates Effective Memory Access Latency*: A compounding factor to the challenge of MSHR sizing is latency fluctuation stemming from dynamic effects in the memory system. Besides the variability arising from the memory architecture itself (§II-B2), even in the absence of a memory system with inherent NUMA characteristics, memory access latency can highly vary as a function of memory system utilization. While nominal memory access latency typically refers to an unloaded system, actual operational latency is highly sensitive to the memory request arrival rate and pattern, as demonstrated in Fig. 3. At high utilization—beyond 60–70% of peak nominal bandwidth—the system enters a non-linear latency increase regime where queuing delays double or triple memory access latency. Hence, expected memory access latency, which is a first-order MSHR provisioning determinant, can undergo drastic temporal fluctuations with changing access patterns and system load conditions.

C. Temporal Variability in MSHR Demand

Besides memory system characteristics, MSHR provisioning decisions are also informed by workload characteristics—particularly the MLP available in the program that is successfully extracted by the CPU. Modern data-intensive workloads, such as AI/ML and large-scale graph analytics often exhibit irregular memory access patterns with alternating phases of pointer-chasing/indirect indexing and high-intensity parallel data retrieval. To illustrate, Fig. 4a shows the distribution of MSHR demand over the execution of various graph and AI/ML workloads, indicating high variability across workloads and a wide spread within the same workload. For example, AI/ML workloads have an average demand of 21 MSHRs, but double that 20–70% of the time. Graph workloads, with a more modest average demand of 6 MSHRs, spend, on average, 11% (and up to 21%) of their execution time requiring more than 16 MSHRs, which is the number of L1 MSHRs in Intel Sapphire Rapids CPUs.

Workloads also exhibit highly variable demand for subentries per MSHR. Fig. 4b illustrates the distribution of allocated subentries per MSHR for various workloads. In contrast to their relatively modest MSHR capacity demands, graph workloads need up to 210 subentries, far exceeding the reported 4–8 subentries available in commercial MSHR implementations [22]. Consequently, certain workloads are highly sensitive to the subentry count supported by the supplied MSHRs.

D. Toward Flexible MSHR Provisioning

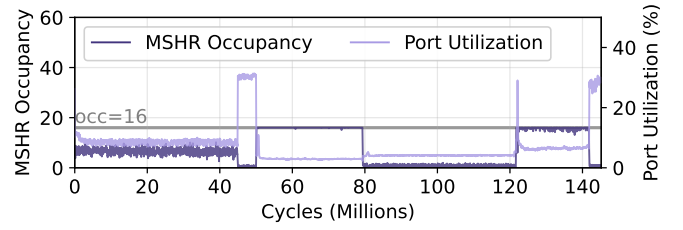
In summary, the confluence of larger CPUs, hierarchical multi-socket topologies, tiered memory systems, and the non-linear effects of variable system load on memory access latency shapes the challenging reality of a higher, less predictable Average Memory Access Time (AMAT). On the software front, phased workload behavior results in highly variable MLP. The combined hardware and software trends require larger MSHR arrays, but also exacerbate the challenge of optimally sizing them. Any static provisioning decision is sub-optimal: too conservative, and the CPU suffers performance loss during phases with high MLP and/or higher AMAT; too generous, and the cost is a power-hungry, large MSHR array, wastefully underutilized during low-MLP phases. We therefore argue for dynamic adjustment of a CPU’s MSHR capacity on demand, by judiciously repurposing existing resources.

III. LEVERAGING UNDERUTILIZED CACHE RESOURCES TO BOOST MEMORY PARALLELISM

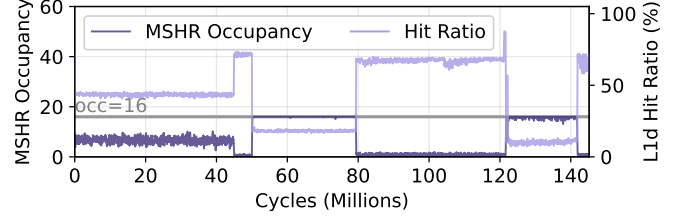
To overcome the limitations of static “one-size-fits-all” MSHR sizing at CPU design time, we advocate a more elastic architecture that can repurpose existing on-chip resources to extend miss tracking capability based on real-time demand. Specifically, we propose *virtualizing* MSHR entries within the already existing storage capacity provided in a typical cache.

The key insight that motivates our approach is that when MSHR usage is high, the utilization of the cache is low, both in terms of bandwidth (i.e., port utilization) and capacity. High MSHR utilization indicates that the application is experiencing a high cache miss ratio, and, conversely, that the contents of the cache’s data array are *currently* of lower value (since they supply fewer cache hits). Fig. 5 illustrates how phases of high MSHR occupancy (50–80M cycles, 120–140M cycles) align with lower port utilization (Fig. 5a), as well as lower hit ratio (Fig. 5b). This relationship between MSHR occupancy and key cache resources (bandwidth and capacity) motivates dynamically repurposing the very resources that are idle (cache ports, cache lines, and their associated data paths) to overcome the structural MSHR capacity limitation when needed.

Building on our insights, we propose a low-overhead mechanism that departs from the rigid design practice of solely relying on dedicated hardware MSHR structures, toward more flexible and dynamic resource management. We introduce *MORIA: MSHR On-demand Reservation with In-cache Allocation*. MORIA’s core idea is the dynamic repurposing of existing cache resources to elastically resize MSHR capacity that seamlessly matches the peak memory access parallelism



(a) Inverse relation of MSHR occupancy and total cache port utilization.



(b) Inverse relation of MSHR occupancy and cache hit ratio.

Fig. 5: Sample timeline from the PageRankDelta workload.

during AMAT or workload MLP spikes. By repurposing existing structures, we provide the performance benefits of a much larger MSHR array without the silicon area and power costs of commensurately large dedicated MSHR hardware.

A. MORIA Overview

MORIA leverages the large capacity and associativity of modern caches as a deep reservoir of additional on-demand MSHR capacity. Using a normal cache line as a temporary MSHR entails storing the metadata required to track the miss, and restoring that information upon the corresponding fill. Notably, an in-cache MSHR allocation does not diminish the cache’s effective capacity. It merely shifts an incoming cache line’s allocation to an earlier point in time: on miss instead of on fill.

A second advantage of MORIA’s design is the seamless transition between miss tracking and data storage. Upon a fill, the incoming data is written directly into the line that was acting as its MSHR. This merging of tracking and storage phases minimizes control logic overhead and reduces the complexity of managing metadata. Finally, a third advantage is that the typical 64B cache line naturally accommodates a large number of subentries, which are in high demand for several workloads, as discussed in §II-C.

In this paper, we study the application of the MORIA mechanism to the L1 data cache. Lower cache levels are typically already provisioned with larger MSHR arrays, thus we expect diminishing performance benefits from MORIA. Nevertheless, our proposed mechanism is equally applicable throughout the cache hierarchy.

IV. DYNAMIC MSHR ALLOCATION WITH MORIA

A. Dual-Mode Cache Line Architecture

MORIA enables each cache line to double as both a data storage unit and a transient, miss metadata unit. The transition between the two states is governed by the *m bit*, a single-bit hardware addition to each tag array entry. Fig. 6 outlines the

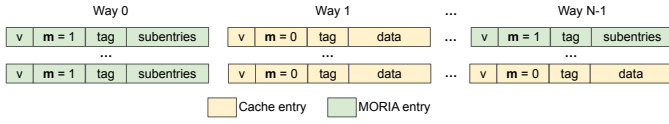


Fig. 6: In MORIA, cache data array entries either hold data (Data Mode) or pending miss information (MSHR Mode).

cache line’s two modes of operation. When the m bit is cleared ($m = 0$), the cache line operates in **Data Mode**. In this state, the tag array line functions as usual, storing the address tag, while the data array line holds the actual cached line. When the m bit is set ($m = 1$), the line operates in **MSHR Mode**. In this state, the line is no longer a valid data source; instead, it serves as a tracking structure for an outstanding memory request.

In MSHR Mode, the corresponding slot in the data array is semantically redefined to accommodate the subentry list of outstanding misses. Each subentry stores the same information as a conventional MSHR subentry: a valid bit, the word offset within the cache line, a specific LSQ ID, acting as a pointer to the instruction in the LSQ that requested the data, and other request format information. Subentry size depends on the representation of recorded metadata; in turn, that size dictates the maximum number of subentries accommodated in the cache line. Typical subentry size in prior work falls in the 15–20 bit range [10], [19], [22], [50], [54].

We adopt a 16-bit subentry format, with the following breakdown: 1 valid bit, 6-bit offset, 8 destination bits (supporting LQ/SQ sizes of up to 256 entries), and 1 bit distinguishing reads from writes. Each cache line in MSHR mode thus supports up to 32 subentries.

We consider two MORIA implementations:

- **Hybrid (MORIA-h)** uses in-cache MSHRs as an overflow mechanism, when the regular hardware MSHRs are depleted.
- **Full In-cache (MORIA-f)** exclusively uses in-cache MSHRs, obviating the need for any regular hardware MSHRs.

In the rest of this section, we detail the functionality and implications of both implementations.

B. Cache Controller Integration

Beyond the tag array extension, MORIA modifies the standard cache pipeline’s logic to handle the interleaved data retrieval and metadata management. The following sections describe the flow of a read request through the modified stages of lookup, allocation, eviction, and eventual data fill. The flow of a write request is similar to that of a read, with some differences in the data fill stage, as detailed in §IV-B3. Our description focuses on MORIA-h, as its functional steps are a superset of the conceptually simpler MORIA-f.

1) *Lookup and Allocation Logic*: Every incoming CPU request initiates a tag lookup that evaluates the valid bit and the m bit in parallel. If the lookup results in a miss (Fig. 7a step ①) and dedicated hardware MSHRs are unavailable ②

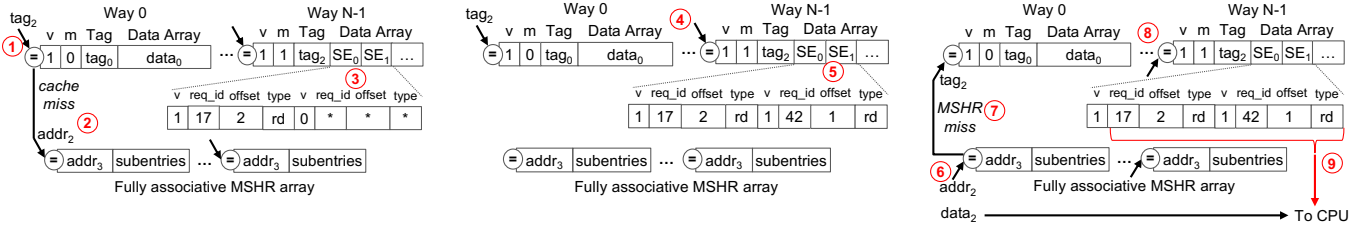
(or entirely absent, in MORIA-f’s case), the in-cache MSHR allocation pipeline is invoked. The cache controller employs the standard replacement policy to identify a victim line to be repurposed as a miss-handling unit, choosing way N-1 in Fig. 7a’s example.

Because cache lines in MSHR Mode contain information necessary to complete an outstanding access, they are considered transient and should not be evicted. MORIA amends the existing replacement policy to only consider lines with $m = 0$ as eviction candidates, protecting cache lines used as MSHRs. Once a victim is selected, its data are evicted as usual and the line becomes logically “locked” (i.e., temporarily not evictable) while it serves as an MSHR for the new address, indicated by an update to the tag array entry that sets the target address and the m bit. Support for similar locking is standard in caches to temporarily block accesses to certain cache lines (e.g., during a fill or a transient coherence state). Simultaneously, the data array is initialized by setting the corresponding subentry’s bit and recording the initial requester’s LSQ ID, word offset, and request type (Fig. 7a step ③).

With MORIA-h, a deadlock may arise when all lines in a set are used as MORIA lines. Because misses using standard MSHRs allocate cache lines on fill, they require an invalid or evictable line upon data return to complete. If an entire set is occupied by MORIA entries, which are temporarily locked in the cache, non-MORIA requests can’t allocate the required storage to complete, blocking the data fill pipeline and stopping any requests from completing. To avoid deadlocks, the cache controller never allows MORIA-h allocations to occupy all the ways of any given cache set. MORIA-f can safely allocate all available ways as MSHRs, as the lack of standard MSHRs precludes such a deadlock scenario.

2) *Secondary Miss Handling*: A cache hit on an entry with $m = 1$, as shown in Fig. 7b, step ④, indicates a hit on a pending miss (i.e., secondary miss). Rather than returning data, as in a normal hit, the cache controller performs a read-modify-write operation on the data array: it checks whether a subentry is available by checking all the subentry valid bits. If all subentries are in use, the request is stalled and the cache is blocked, as in the baseline system on MSHR array depletion. Otherwise, the controller writes the new LSQ ID and other subentry information to the available subentry in the data array line, and sets the corresponding valid bit ⑤.

3) *Eviction and Response Fill*: The eviction pipeline remains unchanged for the victim line, selected at the time of MORIA allocation, following standard writeback protocols. However, the fill pipeline is modified. Upon receiving a response from the lower levels of the memory hierarchy, in the case of MORIA-h, the MSHR array is searched first (Fig. 7c, step ⑥). If there is no match (or no MSHR array, in MORIA-f’s case), a tag array lookup locates the corresponding MORIA line ⑦. The fill logic first reads the data array to extract all accumulated LSQ IDs and request information ⑧ and returns the arriving data to the corresponding LSQ entries ⑨. Finally, the pipeline performs a final update to the tag and data arrays, clearing the m bit and writing the actual memory line into the



(a) Cache state before and after a miss with an in-cache MSHR allocation. A cache miss that finds all standard MSHRs in use allocates a MORIA MSHR in the cache, replacing a victim (e.g., in way N-1).

(b) A cache access hits in a MORIA MSHR cache entry, indicating an already pending miss for the same cache line (i.e., secondary miss). The MORIA MSHR is updated with a new subentry.

(c) Response fill. Valid subentries in the matching MORIA MSHR indicate the parts of $data_2$ to be forwarded to the CPU. After forwarding completes, $data_2$ is installed in the cache's way N-1.

Fig. 7: MORIA-related cache operations for a primary miss, secondary miss, and cache fill.

data array, effectively transitioning the line back to Data Mode for subsequent hits (not shown). If the outstanding request is a write, the data drains from the CPU's store buffer to the data array.

C. Hardware and System Overheads

Enabling MORIA's increased miss-handling concurrency incurs some hardware and cache bandwidth overheads. We categorize these costs into structural modifications to the cache array (§IV-C1) and increase in port pressure (§IV-C2).

1) *Hardware Area, Power and Logic Complexity:* The structural additions required for MORIA are minimal relative to the total cache area. The primary storage overhead is the addition of the m bit to the tag array (Fig. 8). For a typical 48 KB L1 cache with 64-byte lines, this equates to 768 additional bits (one per cache line). MORIA's logic overhead involves the following components:

Status Logic: In an n -way cache, an AND gate per cache way and an n -way OR gate are integrated into the hit/miss detection logic, with the purpose of verifying the m bit status during tag comparison (green path in Fig. 8). An added decoder (not shown in Fig. 8) evaluates the *MSHR?* and *Hit?* signals to determine whether an access is a Miss, a Data Hit or a MORIA Hit. Similar to standard MSHR operation, on a secondary miss to a MORIA line, cache controller logic verifies subentry availability.

Routing: Minimal multiplexing logic is added to redirect data array outputs to either the CPU (in Data Mode), the MSHR subentry availability check logic, or the MSHR fill logic (in MSHR Mode).

We extend McPAT [32] to model these additions and use a 22nm process to evaluate their overheads over a baseline 48KB 8-way L1d cache. MORIA-h incurs a negligible 0.001% area increase. MORIA-f's net effect is a 1.6% area reduction compared to the baseline, due to MSHR array elimination. Although details on commercial MSHRs are unknown, we can estimate the costs of extending the MSHR array. MORIA's 768-bit budget is enough for just one additional MSHR with 8 subentries, a marginal gain compared to the MSHR demands of bandwidth-intensive applications, as established by Fig. 4.

L1d dynamic power consumption increases by 9.8% on average with MORIA-h and by 4.9% with MORIA-f, due to

TABLE I: Additional cache port usage introduced by MORIA.

Case #	Action	Extra cache port usage	
		Read	Write
1	Data Hit	0	0
2	Miss or Fill, with standard MSHR	0	0
3	Miss with MORIA MSHR	0	1
4	MORIA Hit (i.e., secondary miss)	0	1
5	Fill with MORIA MSHR	1	0

the added hardware and the MORIA-related cache accesses. MORIA-h's higher power is due to the additional MSHR array access on every miss and fill, which MORIA-f eschews. We compare MORIA's power and area overheads against expanded MSHR arrays in §VI-B. Overall, MORIA incurs a modest L1d cache power increase for substantial performance gains, as demonstrated in our upcoming evaluation (§VI).

2) *Bandwidth and Port Contention:* Manipulating MORIA lines adds pressure to the read and write ports of the cache to handle outstanding miss metadata. Accesses that do not entail MORIA-related actions (i.e., hits, misses using regular MSHRs) do not affect port utilization (Table I, cases 1–2). Cache misses that use MORIA to hold their miss status handling metadata, however, require additional accesses to the tag and data arrays to retrieve and update miss metadata. All misses using a MORIA line (Table I, cases 3–4) perform one extra write to the cache when the miss is detected, to store the outstanding miss metadata (Fig. 7b). When a response is received for an address tracked by a MORIA line (Table I, case 5), an additional read to the cache is performed to retrieve the metadata required to respond to the processor's pending requests (Fig. 7c). Overall, a miss that employs a MORIA line instead of a regular MSHR incurs an additional cache write (on allocation) and an additional cache read (on fill). The additional read is amortized across multiple secondary misses using the same MORIA line. We evaluate the impact of MORIA's cache port contention in §VI-G.

3) *Cache Access Latency Implications:* The added overhead of MORIA functionality's integration on the cache pipeline's critical path is trivial. On a standard cache lookup, signal propagation through the added AND and OR gates happens in parallel with the default tag-match logic (Fig. 8), therefore not affecting the critical path. The added 2-to-4 decoder to distinguish the outcomes of MSHR and normal

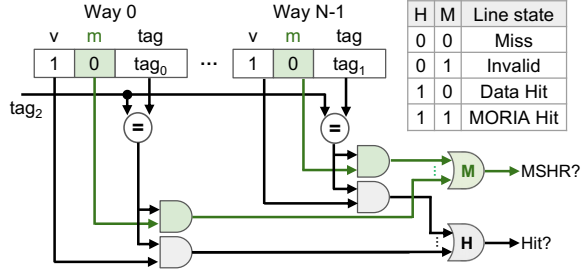


Fig. 8: Cache storage and logic additions for MORIA. Green components indicate our additions to a standard cache’s tag array. The data array remains unmodified.

operation adds $0.44ps$ to the hit path, which, at typical frequencies, does not alter the cache’s cycle time.

During MSHR allocation, MORIA writes miss metadata in the cache instead of the MSHR array, which incurs a higher latency due to the cache’s larger structure. That increased delay does not impact miss latency, as the miss is propagated to lower levels in parallel with the metadata write. However, the added cache port pressure may indirectly delay subsequent cache accesses, which we evaluate in §VI-G.

Each MORIA entry allocation invokes the replacement and eviction policy on cache miss. Shifted earlier in time—on miss, instead of on fill—these actions do not increase miss latency.

D. MORIA-f Versus MORIA-h Considerations

Each of the two MORIA implementations has its distinct merits. On the hardware provisioning front, MORIA-f is a subset of MORIA-h, as both require the same cache extensions, but the former allows the removal of regular MSHRs altogether, reducing power and hardware costs (§IV-C1). In terms of runtime resource consumption, MORIA-h may never employ its capability for in-cache MSHRs—and the associated increased port usage—if the workload’s MSHR demand never exceeds the regular MSHR file’s capacity. MORIA-h’s operation aligns with our insight on the anti-correlation of MSHR and cache port usage (§III). While MORIA-f’s doubling of port usage for all cache misses may appear to be a performance concern, it is rarely so in practice, as we show in §VI-C, given that port utilization is dominated by cache hits. Besides the added port pressure, MORIA support does not introduce overheads on the cache hit path, as discussed in §IV-C3.

Finally, workloads with high subentry requirements, like graph processing (§II-C), favor MORIA-f. Hardware MSHRs typically supply fewer subentries than those a cache line can accommodate (32 in our implementations). Therefore, MORIA-h is more likely to experience CPU stalls and performance degradation due to subentry depletion, when using regular MSHRs. Subentry depletion cannot be alleviated by MORIA-h’s capability of allocating in-cache MSHRs, without considerably increasing its design complexity. In contrast, MORIA-f uniformly enjoys the benefit of plentiful subentries across all available (in-cache) MSHRs. We demonstrate the significance of this implication in our evaluation (§VI-C).

TABLE II: System parameters for simulation in ChampSim.

CPU	x86-64 core modeled after Sapphire Rapids P-core, OoO, 2.28 GHz, 6/8-wide fetch/retirement, 512-entry ROB, 192/114-entry LQ/SQ
L1i Cache	32KB 8-way, 64B lines, 3-cycle access, next-line prefetcher
L1d Cache	48KB 12-way, 64B lines, 5-cycle access, 3 load & 2 store ports, next-line + IP-stride prefetcher
L2 Cache	1.25MB, 10-way, 15-cycle access, non-inclusive, SPP prefetcher [28]
LLC	Inclusive, 2MB/core, 16-way, 60-cycle access, non-inclusive
MSHRs	8/16/48 at L1i/L1d/L2, 64/core at LLC; 8 subentries each
DRAM	Mess-modeled load-latency curve of DDR5-3200 [17], bandwidth availability: 9.6 GB/s $\times$ core count
CXL	Mess-modeled load-latency curve of DDR5-5600 over CXL 2.0 [17], bandwidth availability: 4.8 GB/s $\times$ core count

V. METHODOLOGY

Modeled System. We use the ChampSim simulator [21] to model a core after the Sapphire Rapids P-core microarchitecture [5], [6], and the Mess methodology [17] to model memory with DDR5-3200 load-latency characteristics. We perform single- and multi-core experiments to assess scalability with core count. Both configurations represent a scaled-down version of a Sapphire Rapids CPU, with LLC and memory resources scaled down proportionally to match core count in each case. Table II summarizes the simulation parameters used.

Because MORIA expands the available MSHR resources, it can significantly alter the behavior of hardware prefetchers by allowing more prefetches to be issued. To isolate this effect and ensure a fair comparison between configurations, we limit the upper bound of total pending prefetches per L1d cache to 8. We further discuss prefetching considerations in §VII.

Evaluated System Configurations. We employ five system configurations to highlight MORIA’s operation and performance improvements. All MSHR comparisons refer to the L1 data cache, while the rest of the system remains unmodified.

- *Baseline* only uses the available MSHR file for outstanding misses. Modeled after Sapphire Rapids [6], it features 16 MSHRs with 8 subentries each by default, but we also evaluate different provisioning variants. We use the notation *Baseline*(M,N) to indicate a configuration with M MSHRs and N subentries each, while “Baseline” without a qualifier implies the default *Baseline*(16,8) configuration.
- *MORIA-h* is our proposed hybrid design that allocates in-cache MSHRs when the regular MSHR file is depleted. In-cache MSHRs are limited to 11 (out of 12) ways per set to prevent deadlocks (§IV-B1). We use the notation *MORIA-h*(M,N) to indicate a configuration where MORIA-h is paired with M regular MSHRs featuring N subentries each.
- *MORIA-f* is our proposed design that completely obviates a dedicated MSHR file for exclusive use of in-cache MSHRs. In-cache MSHRs can occupy all ways of a set.
- *MORIA-f-free* is an unrealistic version of MORIA-f that does not consume any of the additional cache port bandwidth required for in-cache MSHR management.
- *Infinite* represents a hypothetical system with an infinite amount of regular MSHRs, each limited to 32 subentries.

Memory Configurations. We examine three memory systems of increasing bandwidth availability and latency variability:

- *DRAM Uniform Memory Access (DRAM-UMA)*: Our simplest memory configuration, modeled after a DDR5-3200 memory module’s load-latency characteristics.
- *DRAM-CXL*: A system with a CXL-based memory expander, modeled after a DDR5-5600 memory module’s load-latency characteristics, with an additional 100ns latency penalty to account for CXL’s overhead. The CXL device offers bandwidth equal to 50% of the primary DDR. We randomly place 30% of pages in the CXL device.
- *DRAM Non-Uniform Memory Access (DRAM-NUMA)*: A multi-socket system with additional memory capacity and bandwidth at a latency premium. The remote memory is another DDR5-3200 memory module, doubling the available memory bandwidth. Accessing the remote memory node incurs a fixed interconnect overhead of 50ns. We randomly place 50% of pages on the remote NUMA node.

The available memory bandwidth is scaled to correspond to the number of cores simulated. Unless stated otherwise, *the presented results refer to the DRAM-CXL memory system.*

Workloads. We evaluate MORIA on workloads from the Ligr Suite [48], SPEC2017 [8], [14], PARSEC [13], [46], and the datacenter and AI/ML benchmarks from DPC4 [1]. We also implement a number of linear algebra microbenchmarks using OpenBLAS [4] to represent compute-intensive workloads with L2 and LLC cache-resident datasets, denoted as *_l2* and *_llc*, respectively, creating high cache port pressure.

We run the selected workloads using their SimPoint traces where available, and generate traces for our OpenBLAS microbenchmarks using ChampSim’s tracing tool. In total, we evaluate 309 traces across 86 benchmarks. Each trace is run for 100 million warm-up instructions and 150 million recorded execution instructions. Final results for each workload are derived from the aggregate statistics of its corresponding SimPoint traces [41].

For multi-core evaluation, we construct 15 heterogeneous workload mixes, each comprising four different workloads. Given the impact of memory contention on resulting AMAT (§II-B3), we construct scenarios that cover a spectrum of memory pressure by selecting the workloads in each mix based on their average memory bandwidth utilization in the single-core *Baseline* setup. We construct five workload mixes for each of the following three categories:

- 1) *Aggressive-All*: All four workloads exhibit high average memory utilization (>50% of their per-core bandwidth share), creating sustained heavy pressure on the memory system and maximum contention for MSHR resources.
- 2) *Aggressive-Single*: One workload exhibits high average memory utilization (>50%) and three maintain low average utilization (<20%), representing colocation of workloads with complementary characteristics.
- 3) *Medium-Low*: All four workloads exhibit medium to low average memory bandwidth utilization (<50%).

Multi-core experiments warm up until each core has completed

100 million instructions. Simulation ends after every core executes 150 million instructions post warm-up. Any trace that ends before the simulation concludes is restarted.

VI. EVALUATION

We begin by comparing MORIA-f against Baseline variants (§VI-A and §VI-B) and compare MORIA-h variants to MORIA-f in §VI-C. §VI-D evaluates area, power, and energy efficiency. §VI-E evaluates MORIA-f under different memory configurations. §VI-F illustrates how MORIA dynamically adjusts to fluctuating MSHR requirements during an application’s execution. §VI-G showcases MORIA’s effect on cache resource availability and operation. Finally, §VI-H analyzes MORIA’s impact on cache hit ratio, and §VI-I evaluates MORIA in multicore systems with workload mixes.

A. MSHR Capacity Bottleneck Mitigation

Fig. 9 presents a comparison of MORIA-f against Baseline(M,8), with $M=\{16, 32, 64\}$. For brevity, Fig. 9 shows individual workload results for three of our six evaluated benchmark suites, and geomean values for all six. §VI-C shows results for all 86 workloads.

Ligr workloads consistently benefit from MORIA-f across all Baseline configurations as MSHR capacity increases, primarily attributable to MORIA-f’s extended subentry capacity rather than raw MSHR count, as evidenced in §VI-B.

AI/ML workloads effectively exploit MORIA-f’s expanded MSHR capacity relative to Baseline(16,8), with MORIA-f’s gains gradually decreasing as Baseline MSHR counts scale to 32 and 64. Notably, certain benchmarks with exceptionally high MSHR requirements, such as *bark*, continue to derive substantial benefits from MORIA-f even when compared against Baseline(64,8).

Several OpenBLAS workloads (e.g., *daxpy_llc*, *dcopy_llc*) similarly leverage MORIA-f’s additional capacity advantages over Baseline(16,8), achieving speedups of up to 2.37 $\times$, with diminishing returns as Baseline’s MSHR counts increase. A subset of OpenBLAS workloads are highly sensitive to cache port bandwidth consumption and, as a result, favor traditional MSHR capacity expansion over MORIA-f, as conventional scaling avoids the port bandwidth overhead inherent to MORIA-f’s design. §VI-C analyzes MORIA-f’s port bandwidth and demonstrates that MORIA-h, which ameliorates port bandwidth demands, curbs the slowdown for some of these workloads.

For most workloads, MORIA-f performs on par or better than expanded MSHR configurations (1.14 $\times$ /1.08 $\times$ /1.06 $\times$ on average over Baseline(16,8)/(32,8)/(64,8)). MORIA-f outperforms traditional MSHR designs with expanded capacity, at a lower area overhead, and while consuming the same (compared to Baseline(32,8)) or less (−3.4% compared to Baseline(64,8)) power (see Table III).

B. MSHR Subentry Bottleneck Mitigation

Fig. 10 evaluates MORIA-f against Baseline(16,N), with $N=\{8, 32, 128\}$ representing typical, iso-subentry (with respect

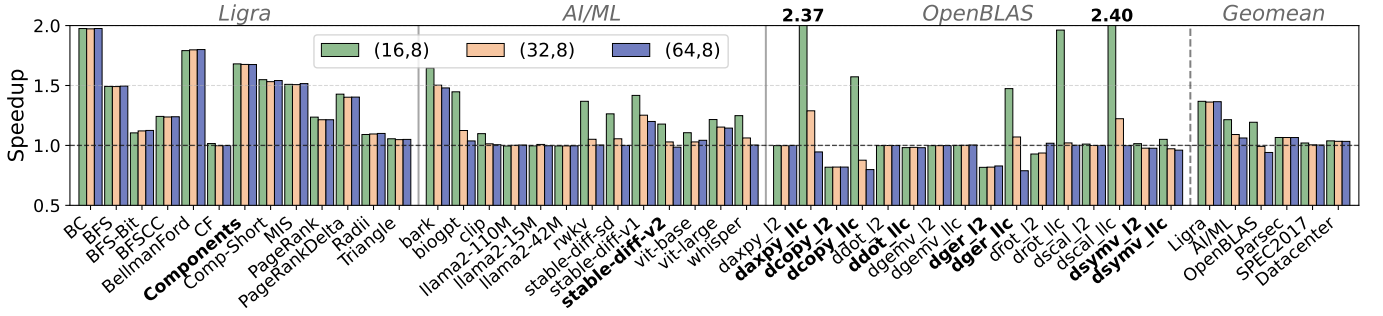


Fig. 9: Performance of MORIA-f compared to Baseline(M,8) when varying Baseline’s number of MSHRs M. Workloads favoring MORIA-h(16,32) over MORIA-f ($workloads_{f < h}$) are marked in bold and are further analyzed in Fig. 12.

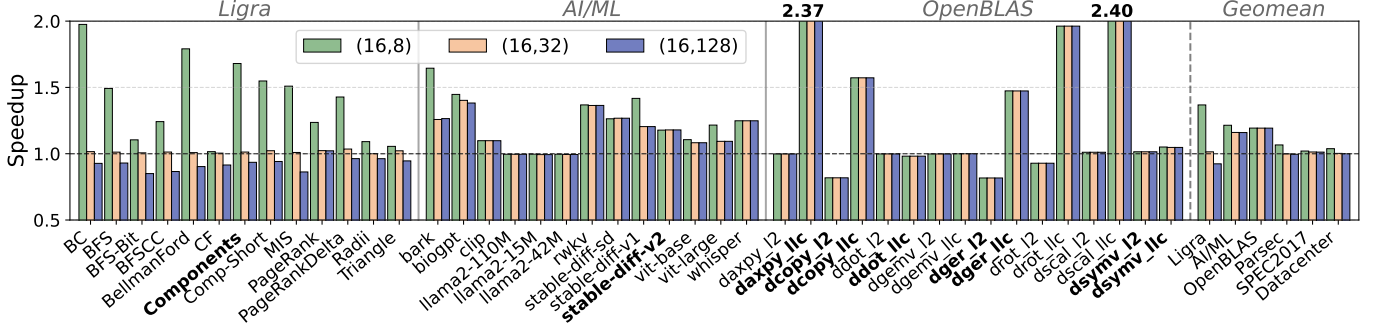


Fig. 10: Performance of MORIA-f compared to Baseline(16,N) when varying Baseline’s number of subentries per MSHR N. Workloads favoring MORIA-h(16,32) over MORIA-f ($workloads_{f < h}$) are marked in bold and are further analyzed in Fig. 12.

to MORIA-f), and highly unrealistic provisioning, respectively. For brevity, Fig. 10 shows individual workload results for three of our six evaluated suites, and geomean values for all six.

Across the subentry-hungry Ligra workloads, MORIA-f yields an average speedup of $1.36\times$ (up to $1.98\times$) against Baseline(16,8). The iso-subentry comparison against Baseline(16,32) isolates the effects of subentry disparity and reveals the fraction of gains that are purely derived from MORIA-f’s expanded MSHR capacity. Speedups for Ligra persist but are modest ($1.01\times$ average, up to $1.04\times$). The (highly unrealistic) Baseline(16,128) offsets MORIA-f’s advantages of expanded MSHR capacity, as Ligra workloads are predominantly subentry-hungry: as indicated in Fig. 4b, even 32 subentries are occasionally insufficient.

MORIA-f benefits AI/ML workloads significantly, yielding average/max speedup of $1.22\times/1.65\times$ over Baseline(16,8). The benefit stems predominantly from additional MSHRs rather than subentries, as a comparison of the Baseline with 8 and 32 subentries reveals. OpenBLAS kernels exhibit a wider range of speedups, with an average of $1.19\times$, up to $2.4\times$, and no sensitivity to subentries. Across all workloads, MORIA-f achieves an average speedup of $1.14\times/1.06\times/1.05\times$ against Baseline with 8/32/128 subentries, respectively.

MORIA-f is rarely inferior to the Baseline with realistic subentry provisioning (8, or even 32). Where slowdowns do occur, they stem from port bandwidth overhead (§VI-C)—an issue MORIA-h partially curbs by reducing these demands.

C. MORIA-f and MORIA-h Comparison

Fig. 11 compares MORIA-f to two MORIA-h(16,N) variants, where (16,N) refers to (entries, subentries) of the hardware MSHR array that is used before falling back to in-cache MSHR allocation. Results are shown as stacked bars, because MORIA-h(16,32)’s performance is strictly superior to MORIA-h(16,8). We first examine the relationship between MORIA-f and MORIA-h(16,8), which is a realistic subentry provisioning for regular MSHRs. We refer to workloads where MORIA-f outperforms MORIA-h(16,8) as $workloads_{f > h}$. Most of $workloads_{f > h}$, most notably Ligra, exhibit high subentry pressure, as seen in §VI-B. Similar to Baseline(16,8), the limited subentry count in MORIA-h(16,8)’s regular MSHRs incurs stalls, blocking the ability of using additional in-cache MSHRs. In contrast, MORIA-f’s uniform availability of 32 subentries per MSHR alleviates this bottleneck, outperforming MORIA-h(16,8) by $1.37\times$ on average for Ligra workloads and by $1.07\times$ on average (up to $1.98\times$) across all workloads. MORIA-h(16,32)’s superiority over MORIA-h(16,8), especially in subentry-heavy workloads like Ligra, confirms the subentry limit of regular MSHRs as a major performance determinant for MORIA-h.

MORIA-h outperforms MORIA-f for a small minority of workloads ($workloads_{f < h}$), mostly OpenBLAS kernels, by up to $1.22\times$. MORIA-f inherently consumes more port bandwidth, as all MSHR allocations incur additional cache accesses, and forces earlier cache line evictions. The compound effect is that MORIA-h can yield superior performance

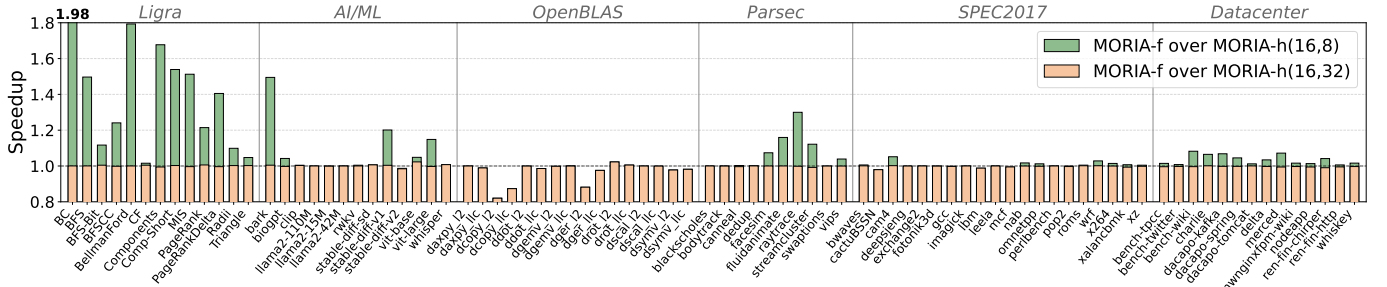


Fig. 11: Performance of MORIA-f normalized to MORIA-h(16,N), where N indicates the number of subentries per regular MSHR. All in-cache MSHRs provide 32 subentries.

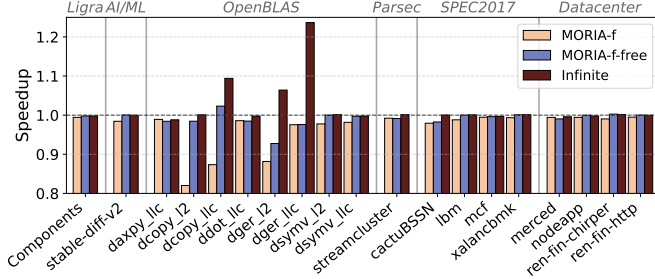


Fig. 12: Performance of MORIA-f, MORIA-f-free, and Infinite for $workloads_{f<h}$, normalized to MORIA-h(16,32).

to MORIA-f, although for a notably small fraction of our evaluated workloads.

For example, `dcopy_12` has a modest need for additional MSHRs (14% of time with occupancy > 16), but very high read port utilization. Therefore, MORIA-f’s added port pressure causes slowdown ($0.82\times$). MORIA-h would rarely allocate additional in-cache MSHRs, minimizing excess port utilization. Exceptionally, a few workloads are so sensitive to port pressure that even MORIA-h would underperform relative to Baseline. For example, `dger_12` experiences a slowdown with both MORIA-f ($0.82\times$) and MORIA-h ($0.93\times$).

To quantify the impact of MORIA-f’s overheads, Fig. 12 evaluates the MORIA-f-free and *Infinite* configurations against MORIA-h(16,32) for $workloads_{f<h}$. In workloads where excess port pressure was the dominant source of MORIA-f’s disadvantage (e.g., AI/ML, `dsymv` kernels), MORIA-f-free matches MORIA-h(16,32)’s performance (i.e., $speedup \approx 1$). Nevertheless, a few workloads continue to favor MORIA-h(16,32) over MORIA-f-free, indicating that port bandwidth alone does not fully account for MORIA-f’s shortcomings.

Infinite, which additionally removes MORIA’s effect of earlier evictions, performs on par with (or better than) MORIA-h(16,32), confirming that the altered timing of evictions can impact MORIA-f’s performance. This is particularly evident in cache-resident OpenBLAS kernels, some of which are highly sensitive to L1 locality.

To summarize, *across most workloads*, MORIA-f and MORIA-h perform equivalently, as long as subentries in MORIA-h’s regular MSHRs are amply provisioned. MORIA-h holds an advantage in workloads where port bandwidth pressure and L1 cache locality are jointly critical, and MORIA-f

TABLE III: MSHR array expansion compared to Baseline (16,8). (M,N) denotes M MSHRs with N subentries each. L1d area and power overheads are derived from McPAT.

Configuration	Dedicated MSHRs	In-Cache MSHRs	L1d Area (%)	L1d Power (%)	Speedup over (16,8)
Baseline	(32,8)	-	+0.05	+4.9	1.06
	(64,8)	-	+0.16	+8.3	1.07
	(16,32)	-	+9.4	+11.2	1.07
	(32,32)	-	+9.6	+18.1	1.14
	(64,32)	-	+9.9	+22.1	1.15
MORIA-h	(16,8)	(704,32)	+0.001	+9.8	1.07
MORIA-f	-	(768,32)	-1.6	+4.9	1.14

is unequivocally superior when subentry capacity is critical. While MORIA-f is not universally superior, it largely performs on par with MORIA-h(16,32) across most workloads, except the most port-bandwidth- and cache-locality-sensitive cases. Most of Fig. 12’s $workloads_{f<h}$ (bolded in Figs. 9 and 10) still benefit from MORIA-f over Baseline, some significantly. A MORIA-h implementation would further accelerate these workloads. In light of our analysis, the rest of our evaluation focuses on MORIA-f for brevity.

D. Power and Area Overheads

Table III summarizes MORIA-f’s performance and overheads alongside conventional designs with expanded MSHR arrays both in terms of MSHR and subentry capacity. As discussed in §VI-A, MORIA-f remains superior to Baseline configurations with $2 - 4\times$ more MSHRs when subentries remain at 8. When jointly increasing MSHRs and their subentry capacity, Baseline performs *on par* with MORIA-f. However, overprovisioning MSHRs and subentries incurs substantial L1 power and area increase. For instance, MORIA-f achieves the same average speedup as Baseline(32,32), with 11% lower area and 13% lower power overhead.

E. Sensitivity to Memory Configuration

While we center our evaluation around the *DRAM-CXL* memory system configuration, Fig. 13 provides a summary of results, across all 86 workloads, for all three memory configurations we considered. Individual workload trends in other memory systems are qualitatively identical to those discussed in §VI-A–§VI-C for the *DRAM-CXL* memory system and are therefore omitted for brevity.

As the available memory bandwidth and its latency (average and variability) grow from *DRAM-UMA* to *DRAM-CXL* and *DRAM-NUMA*, MORIA-f’s average speedup also increases from $1.11\times$ to $1.14\times$ and $1.15\times$, respectively. MORIA-f dynamically and transparently adjusts MSHR availability to the memory system’s effective bandwidth-delay product, allowing each workload to convert its inherent MLP to runtime parallelism, overcoming the Baseline’s static design-time provisioning limitations.

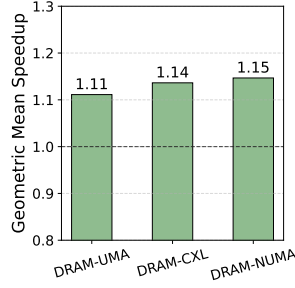


Fig. 13: Speedup of MORIA-f over Baseline for various memory systems.

F. Temporal Fluctuations in MSHR Demand

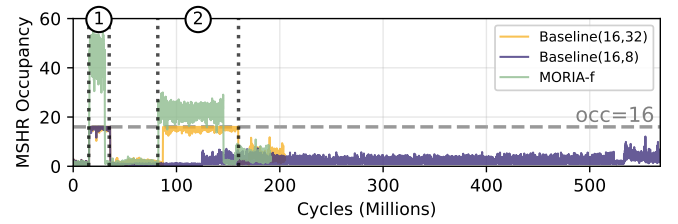
To illustrate how MORIA dynamically adjusts the number of available MSHRs during execution, Fig. 14 shows the temporal behavior of the PageRankDelta workload during a sample execution trace of 150 million instructions. We compare MORIA-f against two Baseline variants, with 8 and 32 subentries per MSHR.

Fig. 14a’s timeline of total MSHR occupancy (regular and in-cache) indicates that the fixed hardware limit of 16 dedicated MSHRs with 8 subentries suffices most of the time. However, the workload exhibits high-intensity intervals with MLP exceeding these hardware limits. In phase ①, the number of hardware MSHRs becomes a performance bottleneck. By re-purposing L1d cache lines, MORIA-f allocates up to 40 additional MSHRs, almost quadrupling the cache miss concurrency beyond the 16-entry dedicated MSHR limit.

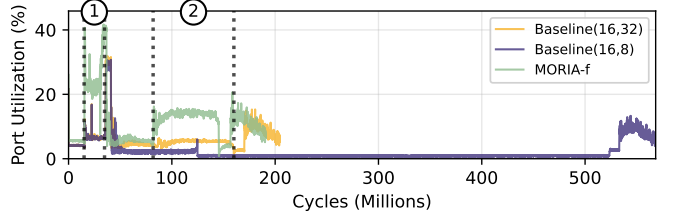
In phase ②, both MSHR and subentry capacity limitations hurt performance. Baseline(16,8) cannot support the workload’s subentry requirements, keeping occupancy low and taking more than 300 million additional cycles to complete the same set of instructions. Baseline(16,32), in comparison, removes the subentry bottleneck, increasing MSHR occupancy and completing the instruction trace $2.75\times$ faster than Baseline(16,8). However, it remains bounded by its 16 MSHRs. MORIA-f removes both bottlenecks, by dynamically expanding MSHR occupancy as needed. With enhanced MSHR and subentry capacity, the system fully exploits the workload’s MLP, resulting in a speedup of $2.92\times$ over Baseline(16,8) and $1.06\times$ over Baseline(16,32) for Fig. 14’s interval. MORIA-f exhibits high sensitivity to workload fluctuations, modulating the number of allocated lines in direct response to the dynamic requirements of the workload and the memory subsystem.

G. Effect on Cache Resources

MORIA’s effectiveness rests on the insight that periods of high miss ratio also exhibit low port utilization. Fig. 14b presents the aggregated read and write port bandwidth utilization for the L1d cache during the same execution interval as Fig. 14a. The phases of elevated miss ratio and increased MSHR demand have ample available port bandwidth



(a) MSHR occupancy.



(b) Total (read+write) L1d port bandwidth utilization.

Fig. 14: Execution timeline for PageRankDelta.

to accommodate MORIA-f’s operational requirements. While MORIA-f causes a notable increase in cache port usage during periods of high in-cache MSHR usage (phases ① and ②), port bandwidth utilization remains well under saturation.

Expanding our analysis to multiple workloads, Fig. 15 illustrates the average L1d port utilization by Baseline and MORIA-f. Most workloads exhibit low Baseline port utilization (15.88% on average), confirming that MORIA-f’s bandwidth overhead (additional 8.29% on average) does not manifest as a significant performance bottleneck. OpenBLAS are the most port-bandwidth-hungry workloads due to their high MLP and cache locality. Despite the increase in port pressure, almost all workloads experience a reduction in average hit latency with MORIA-f (Fig. 15, over-bar numbers).¹ The expanded MSHR capacity allows misses to be issued faster, unblocking hits that were queued behind them. Only two workloads experience a minor hit latency increase.

Fig. 16 sheds light into MSHR allocation dynamics with a breakdown of allocation attempt outcomes for Baseline and MORIA-f. We focus on the subset of workloads where at least 10% of allocation attempts fail due to port unavailability (*workloads_{high_bw}*). Failed allocation attempts are broken down into their various causes: insufficient MSHR capacity, insufficient subentry capacity, unavailable port bandwidth, and backpressure from lower cache levels (“Lower Q” in Fig. 16’s legend). Most workloads experiencing allocation stalls due to port contention with MORIA-f still enjoy a reduction in MSHR allocation failures, often leading to speedup. In a few cases, like *dcopy_l2*, port contention increases the total of failed allocations compared to Baseline, resulting in performance degradation as explained in §VI-B.

H. Effect on Cache Hit Ratio

¹For loads that hit in the L1 cache, we define hit latency as the time between when the request is ready to be issued from the load queue to the memory hierarchy (i.e., all its data dependencies have been satisfied), until a response is returned to the CPU’s load queue.

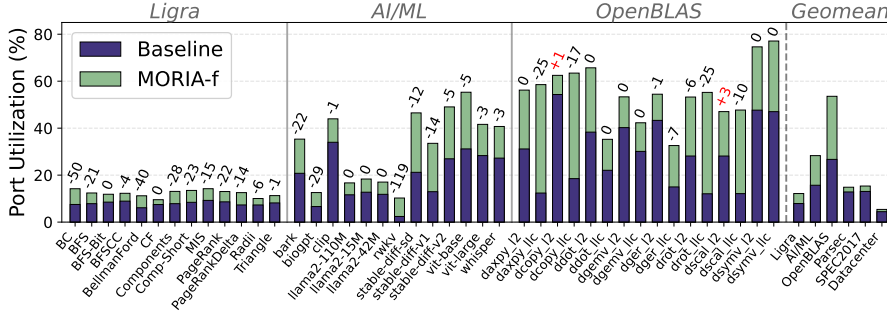


Fig. 15: Average L1d port (read+write) bandwidth utilization for Baseline and MORIA-f. Over-bar numbers show the average hit latency change (in cycles) achieved with MORIA-f. Hit latency increases are rare and are highlighted in red.

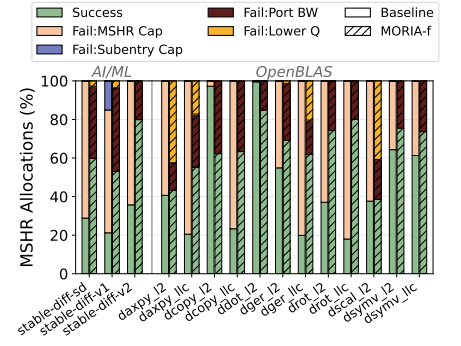


Fig. 16: MSHR allocation breakdown with Baseline and MORIA-f for *workloads_{high_bw}*.

TABLE IV: L1d hit ratio change relative to Baseline(16,8). Reported values are percentage changes.

	Ligra	AI/ML	OpenBLAS	Parsec	SPEC 2017	Datacenter
MORIA-f	-13.9	-3.6	-3.3	-1.1	-1.0	-2.0
Baseline(16,32)	-11.1	-0.5	0.0	-1.8	-0.2	-1.3

MORIA’s earlier cache replacement decisions (on miss rather than on fill) may affect the cache hit ratio. Table IV reports the relative change in average L1d hit ratio against Baseline(16,8). Most workloads experience modest reductions below 3.6%, with Ligra suffering the largest impact (13.9%). However, Ligra’s substantial hit ratio reduction does not translate to performance loss. In Baseline(16,8), accesses arriving while all subentries for a pending primary miss are occupied are stalled. By the time they are issued, the primary miss has already been served (i.e., the cache line is L1-resident), so they are counted as hits. In contrast, MORIA’s 32 subentries allow these accesses to be issued promptly and counted as secondary misses, which is accounted as a lower hit ratio.

We support this claim with two sources of evidence. First, the reduction in hits between Baseline(16,8) and MORIA-f corresponds closely to an increase in secondary misses, with primary miss counts remaining largely unchanged. Second, Baseline(16,32), which mitigates the same subentry bottleneck, shows a similar hit-ratio reduction to MORIA-f, particularly for Ligra (11.1% vs. 13.9%). The remaining small gap partly reflects the additional early-eviction cost unique to MORIA’s allocate-on-miss policy. Despite the lower hit ratio, MORIA-f reduces average L1d hit latency (see over-bar numbers in Fig. 15) and improves performance.

I. Multicore and Workload Mixes

Fig. 17 shows MORIA-f’s speedup over Baseline(16,8) for 15 workload mixes on a 4-core system. Memory bandwidth availability is 4× higher than the single-core system. We use the notation (workload mix #, bar #) to refer to individual workloads. MORIA-f is beneficial for all evaluated workload mixes, yielding an aggregate geomean speedup of 1.2×.

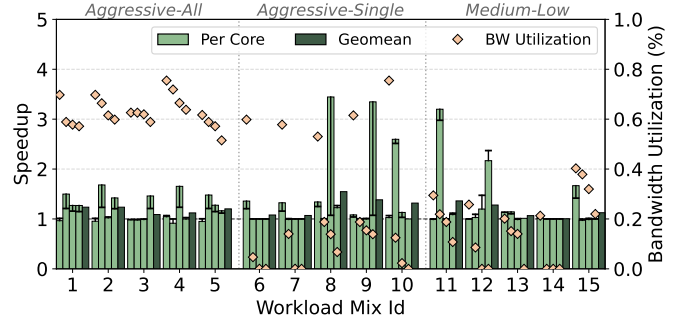


Fig. 17: MORIA-f speedup over Baseline for 4-core workload mixes. Workloads within each mix are ordered by decreasing average memory bandwidth utilization in the single-core Baseline. Error bars indicate the workload’s corresponding MORIA-f speedup in the single-core experiments.

Many workloads exhibit much higher per-core speedups when run colocated compared to their single-core counterparts. We identify two different reasons for such phenomenon. In the first category, like (8, 3) and (9, 4), interference from colocated workloads in the form of bandwidth and LLC pressure degrades the workload’s performance in the Baseline system. MORIA-f allows the workload to recover its original performance attained in isolation, by aggressively expanding its memory access parallelism. In the second category, like (2, 2), the workload leverages MORIA-f to use more bandwidth than its “fair share” (i.e., > 25% of the total bandwidth).

In a few mixes, one workload of the group experiences slowdown, as a combined effect of (i) memory latency sensitivity, (ii) sufficiently low MLP to not benefit from MORIA-f’s higher MSHR availability, and (iii) being penalized by the aggressive memory behavior of their neighbors (exacerbated by MORIA-f), which pollutes the LLC and increases memory latency due to contention. Despite such outliers, MORIA-f delivers substantial aggregate gains across all mixes.

VII. DISCUSSION

Historical Perspective of In-Cache MSHRs. In-cache MSHRs were considered as a design point in the 90s [19], but were deemed impractical due to microarchitectural constraints.

Farkas and Jouppi argued that embedding MSHRs within the cache would significantly increase port pressure, as both demand accesses and miss-handling operations would contend for the same tag/data arrays, a serious limitation in early caches with few, narrow ports. Additionally, the—prevalent at the time—small direct-mapped L1 caches could result in early blocking upon the first conflict for in-cache MSHR allocation. Farkas and Jouppi note that while higher associativity could overcome that limitation, it would likely also eliminate the conflict misses that motivate a design supporting more in-flight misses in the first place. Hence, the idea of in-cache MSHRs was not pursued beyond a high conceptual level and was summarily abandoned three decades ago.

While MORIA’s core concept is the same, we revisit that design option under very different technological realities (larger caches with high associativity and many ports), and, unlike prior work [19], present a complete hardware design covering the dual-mode cache line architecture and the full modified cache controller pipeline; a characterization of implementation requirements including port pressure and subentry sizing; and a comprehensive evaluation. Ultimately, our study reaches the opposite conclusion: that *in-cache MSHRs are a viable, elastic, high-performance, and future-proof mechanism to accommodate high memory parallelism across memory systems*.

Hardware Prefetching Considerations. MORIA enables high MSHR expansion capability, which can implicitly enable more aggressive prefetching. Such capability may further boost performance gains afforded by MORIA, or may result in excess bandwidth pressure and cache pollution. While we intentionally isolated such effects in our evaluation by setting a fixed limit to prefetches across configurations (§V), this is an area warranting future study.

VIII. RELATED WORK

MSHR Organization. The concept of non-blocking caches and MSHRs originates from early work by Kroft [30], who introduced lockup-free caches that could support multiple outstanding misses. Farkas and Jouppi [19] explored the design space of MSHRs, contrasting implicit and explicit organizations and demonstrating the importance of flexible subentry allocation. Tuck et al. [50] proposed a hierarchical MSHR organization that combines small per-bank MSHRs with a larger shared global pool. While such hierarchies facilitate scalability, empirical evidence indicates that modern CPUs implement capacity-limited MSHR organizations, limiting MLP.

Scheurich and Dubois [47] first extended MSHRs to cover multicore systems. Jahre and Natvig [25] showed that over-provisioned MSHRs can lead to memory congestion and introduced a mechanism to dynamically limit the available L1 MSHRs of a core to limit memory bandwidth interference. While we did not encounter such phenomena, MORIA’s MSHR allocation flexibility may benefit from such congestion management mechanisms, especially if prefetching limits are increased. Valsan et al. [52] explored inter-core contention on the shared LLC MSHRs. We focused on L1d MSHRs, but MORIA is applicable to all cache levels.

Recent research on MSHR architectures has focused primarily on massively-parallel systems, such as GPUs and FPGAs. Gu and Chen [22] study MSHR and subentry exhaustion in GPUs, and propose forming MSHRs by dynamically linking available slots from a pool on-demand. Asiatici and Ienne [9], [10] propose FPGA-based architectures capable of sustaining thousands of outstanding misses through large, hash-indexed MSHR structures and flexible subentry management. MiCache [54] is an MSHR-inclusive cache that dynamically shares cache storage between data and MSHRs. While MORIA shares the same insights, MiCache applies them in the FPGA space by completely redesigning the cache and its access pipeline. In contrast, MORIA virtualizes additional MSHR capacity with minimal extensions to standard CPU caches.

MLP Extraction Techniques. A variety of techniques aim at extracting more MLP from memory-bound workloads. Simultaneous Multithreading (SMT) [18], [33], [51] and Virtualized Multithreading [20] expose more MLP to the hardware by filling stall cycles with work from co-scheduled threads; they improve CPU throughput, but, unlike MORIA, not single-thread performance. Runahead execution [23], [36], [37], [38], [44], [45] speculatively executes ahead of a stalling load to uncover and prefetch future long-latency misses. Similarly, hardware prefetchers [12], [28], [39], [40] aim to issue speculative memory requests ahead of their use, hiding latency by keeping the memory subsystem occupied before the core actually stalls. On the software side, coroutine-based techniques [26], [29], [34], [42], [43], [53] overlap multiple long-latency accesses from different software contexts on a single core. Whether in software or in hardware, all of these techniques attempt to extract more MLP, and are fundamentally bounded by the number of available MSHRs. MORIA is complementary and synergistic with all of them: by drastically expanding MSHR capacity at minimal cost, it amplifies the effectiveness of any MLP-boosting technique.

IX. CONCLUSION

Modern memory-intensive workloads are frequently bottlenecked by L1 MSHR exhaustion, limiting achievable memory-level parallelism. We introduced MORIA, a novel microarchitectural technique that virtualizes MSHR functionality within the cache to provide scalable, on-demand miss handling. By replacing or augmenting dedicated MSHRs with in-cache miss handling state, MORIA introduces new dynamic resource-repurposing flexibility that eliminates MSHR-induced stalls with minimal hardware overhead, unlocking single-thread performance gains of up to $2.37\times$ (average $1.14\times$).

ACKNOWLEDGMENTS

We thank the anonymous reviewers, Albert Cho, Boris Grot, Pulkrit Gupta, Moin Qureshi, Hamed Seyedroudbari, Anirudh Sarma, Peidi Song, and Georgios Vavouliotis for their feedback and suggestions that helped improve the paper. This work was supported by NSF grant 2237434.

REFERENCES

- [1] “4th Data Prefetching Championship (DPC4).” [Online]. Available: <https://sites.google.com/view/dpc4-2026/home>
- [2] “7-Zip LZMA Benchmark, Intel Westmere.” [Online]. Available: <https://www.7-cpu.com/cpu/Westmere.html>
- [3] “Intel Xeon 6 6700E Sierra Forest Shatters Xeon Expectations.” [Online]. Available: <https://www.servethehome.com/intel-xeon-6-6700e-sierra-forest-shatters-xeon-expectations/>
- [4] “OpenBLAS, An optimized BLAS library.” [Online]. Available: <http://www.openmathlib.org/OpenBLAS/>
- [5] “Popping the Hood on Golden Cove.” [Online]. Available: <https://chipsandcheese.com/p/popping-the-hood-on-golden-cove>
- [6] “Sapphire Rapids: Golden Cove Hits Servers.” [Online]. Available: <https://chipsandcheese.com/p/a-peek-at-sapphire-rapids>
- [7] “Software Optimization Guide for the AMD Zen4 Microarchitecture.” [Online]. Available: https://www.numberworld.org/blogs/2024_8_7_zen5_avx512_tear-down/57647_zen4_sog.pdf
- [8] “Spec2017 Benchmarks.” [Online]. Available: <https://www.spec.org/cpu2017/>
- [9] M. Asiatici and P. Jenne, “Large-scale graph processing on fpgas with caches for thousands of simultaneous misses,” in *Proceedings of the 48th International Symposium on Computer Architecture (ISCA)*, 2021.
- [10] —, “Request, coalesce, serve, and forget: Miss-optimized memory systems for bandwidth-bound cache-unfriendly applications on fpgas,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 2, 2022.
- [11] T. Bang, N. May, I. Petrov, and C. Binnig, “The full story of 1000 cores,” *The VLDB Journal*, vol. 31, pp. 1185–1213, 2022. [Online]. Available: <https://doi.org/10.1007/s00778-022-00742-4>
- [12] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, “Pythia: A customizable hardware prefetching framework using online reinforcement learning,” in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021.
- [13] C. Bienia, S. Kumar, J. P. Singh, and K. Li, “The parsec benchmark suite: characterization and architectural implications,” in *Proceedings of the 17th International Conference on Parallel Architecture and Compilation Techniques (PACT)*, 2008.
- [14] D. Chasapis, “Spec cpu 2017 champsim traces,” Apr. 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.10960004>
- [15] A. Cho and A. Daglis, “StarNUMA: Mitigating NUMA challenges with memory pooling,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.
- [16] A. Cho, A. Saxena, M. Qureshi, and A. Daglis, “COAXIAL: A cxl-centric memory system for scalable servers,” in *Proceedings of the 2024 ACM/IEEE Conference on Supercomputing (SC)*, 2024.
- [17] P. Esmaili-Dokht, F. Sgherzi, V. S. Girelli, I. Boixaderas, M. Carmin, A. Monemi, A. Arnejach, E. Mercadal, G. Llort, P. Radojkovic, M. Moretó, J. Giménez, X. Martorell, E. Ayguadé, J. Labarta, E. Con-falonieri, R. Dubey, and J. Adlard, “A Mess of Memory System Benchmarking, Simulation and Application Profiling,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.
- [18] S. Eyerman and L. Eeckhout, “A memory-level parallelism aware fetch policy for smt processors,” in *Proceedings of the 13th IEEE Symposium on High-Performance Computer Architecture (HPCA)*, 2007.
- [19] K. I. Farkas and N. P. Jouppi, “Complexity/performance tradeoffs with non-blocking loads,” in *Proceedings of the 21st International Symposium on Computer Architecture (ISCA)*, 1994.
- [20] J. Feliu, A. Naithani, J. Sahuquillo, S. Petit, M. Qureshi, and L. Eeckhout, “Vmt: Virtualized multi-threading for accelerating graph workloads on commodity processors,” *IEEE Transactions on Computers*, vol. 71, no. 6, 2021.
- [21] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. Pugsley, and J. Kim, “The championship simulator: Architectural simulation for education and competition,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.14324>
- [22] Y. Gu and L. Chen, “Dynamically linked mshrs for adaptive miss handling in gpus,” in *Proceedings of the ACM International Conference on Supercomputing (ICS)*, 2019.
- [23] M. Hashemi, O. Mutlu, and Y. N. Patt, “Continuous runahead: Transparent hardware acceleration for memory intensive workloads,” in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.
- [24] C. Imes, S. Hofmeyr, D. I. D. Kang, and J. P. Walters, “A case study and characterization of a many-socket, multi-tier numa hpc platform,” in *IEEE/ACM 6th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC) and Workshop on Hierarchical Parallelism for Exascale Computing (HiPar)*, 2020.
- [25] M. Jahre and L. Natvig, “A high performance adaptive miss handling architecture for chip multiprocessors,” in *Transactions on High-Performance Embedded Architectures and Compilers IV*, 2011.
- [26] C. Jonathan, U. F. Minhas, J. Hunter, J. Levandoski, and G. Nishanov, “Exploiting coroutines to attack the” killer nanoseconds,” *Proceedings of the VLDB Endowment*, vol. 11, no. 11, 2018.
- [27] D. K. Kadiyala and A. Daglis, “Pushing the Memory Bandwidth Wall with CXL-enabled Idle I/O Bandwidth Harvesting,” *CoRR*, vol. abs/2511.12349, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2511.12349>
- [28] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti, “Path confidence based lookahead prefetching,” in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.
- [29] O. Kocberber, B. Falsafi, and B. Grot, “Asynchronous memory access chaining,” *Proc. VLDB Endow.*, vol. 9, no. 4, p. 252–263, Dec. 2015.
- [30] D. Kroft, “Lockup-free instruction fetch/prefetch cache organization,” in *Proceedings of the 8th International Symposium on Computer Architecture (ISCA)*, 1981.
- [31] C. Lameter, “An overview of non-uniform memory access,” *Commun. ACM*, vol. 56, no. 9, pp. 59–54, Sep. 2013.
- [32] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, “Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures,” in *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2009.
- [33] C.-K. Luk, “Tolerating memory latency through software-controlled pre-execution in simultaneous multithreading processors,” in *Proceedings of the 28th International Symposium on Computer Architecture (ISCA)*, 2001.
- [34] Z. Luo, S. Son, S. Ratnasamy, and S. Shenker, “Harvesting memory-bound CPU stall cycles in software with MSH,” in *Proceedings of the 18th Symposium on Operating System Design and Implementation (OSDI)*, 2024.
- [35] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. O. Kanaujia, and P. Chauhan, “TPP: transparent page placement for cxl-enabled tiered-memory,” in *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXVIII)*, 2023.
- [36] O. Mutlu, J. Stark, C. Wilkerson, and Y. N. Patt, “Runahead execution: An alternative to very large instruction windows for out-of-order processors,” in *Proceedings of the 9th IEEE Symposium on High-Performance Computer Architecture (HPCA)*, 2003.
- [37] A. Naithani, S. Ainsworth, T. M. Jones, and L. Eeckhout, “Vector runahead,” in *Proceedings of the 48th International Symposium on Computer Architecture (ISCA)*, 2021.
- [38] A. Naithani, J. Feliu, A. Adileh, and L. Eeckhout, “Precise runahead execution,” in *Proceedings of the 26th IEEE Symposium on High-Performance Computer Architecture (HPCA)*, 2020.
- [39] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals-Yúfera, and A. Ros, “Berti: an accurate local-delta data prefetcher,” in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022.
- [40] S. Pakalapati and B. Panda, “Bouquet of instruction pointers: Instruction pointer classifier-based spatial hardware prefetching,” in *Proceedings of the 47th International Symposium on Computer Architecture (ISCA)*, 2020.
- [41] E. Perelman, G. Hamerly, M. Van Biesbrouck, T. Sherwood, and B. Calder, “Using simpoint for accurate and efficient simulation,” in *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, 2003.
- [42] G. Psaropoulos, T. Legler, N. May, and A. Ailamaki, “Interleaving with coroutines: a systematic and practical approach to hide memory latency in index joins,” *The VLDB Journal*, vol. 28, no. 4, 2019.
- [43] G. Psaropoulos, I. Oukid, T. Legler, N. May, and A. Ailamaki, “Bridging the latency gap between nvme and dram for latency-bound operations,” in *Proceedings of the 15th International Workshop on Data Management on New Hardware*, 2019.

- [44] T. Ramirez, A. Pajuelo, O. J. Santana, and M. Valero, "Runahead threads to improve smt performance," in *Proceedings of the 14th IEEE Symposium on High-Performance Computer Architecture (HPCA)*, 2008.
- [45] J. Roelandts, A. Naithani, S. Ainsworth, T. M. Jones, and L. Eeckhout, "Scalar vector runahead," in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.
- [46] SAFARI Research Group, "Parsec 2.1 traces for champsim," Dec. 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.14268119>
- [47] C. Scheurich and M. Dubois, "The design of a lockup-free cache for high-performance multiprocessors," in *Proceedings of the 1988 ACM/IEEE Conference on Supercomputing (SC)*, 1988.
- [48] J. Shun and G. E. Blelloch, "Ligra: a lightweight graph processing framework for shared memory," in *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2013.
- [49] Y. Sun, Y. Yuan, Z. Yu, R. Kuper, C. Song, J. Huang, H. Ji, S. Agarwal, J. Lou, I. Jeong, R. Wang, J. H. Ahn, T. Xu, and N. S. Kim, "Demystifying CXL memory with genuine cxl-ready systems and devices," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.
- [50] J. Tuck, L. Ceze, and J. Torrellas, "Scalable cache miss handling for high memory-level parallelism," in *Proceedings of the 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2006.
- [51] E. Tune, R. Kumar, D. M. Tullsen, and B. Calder, "Balanced multithreading: Increasing throughput via a low cost multithreading hierarchy," in *Proceedings of the 37th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2004.
- [52] P. K. Valsan, H. Yun, and F. Farshchi, "Taming non-blocking caches to improve isolation in multicore real-time systems," in *Proceedings of the 2016 Real-Time and Embedded Technology and Applications Symposium*, 2016.
- [53] M. Vemmou and A. Daglis, "Cosplay: Leveraging task-level parallelism for high-throughput synchronous persistence," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021.
- [54] S. Xu, S. Lu, Z. Shao, X. Liao, and H. Jin, "Micache: An mshr-inclusive non-blocking cache design for fpgas," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2024.